UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+

Student Name: Swayam **UID:** 25MCA20015
Branch: M.C.A **Section/Group:** MCA-1 A
Semester: 2 **Date of Performance:**12/01/2026
Subject Name: TECHINCAL SKILLS **Subject Code:**

AIM: To implement conditional decision-making logic in PostgreSQL using IF–ELSE constructs and CASE expressions for classification, validation, and rule-based data processing.

S/W Requirement: Oracle Database Express Edition and pgAdmin

OBJECTIVES:

- To understand conditional execution in SQL
- To implement decision-making logic using CASE expressions
- To simulate real-world rule validation scenarios
- To classify data based on multiple conditions
- To strengthen SQL logic skills required in interviews and backend systems

Practical / Experiment Steps

Step 1: Database and Table Preparation

Students should first create a table that stores:

- A unique identifier
- A schema or entity name
- A numeric count representing violations or issues

Populate the table with multiple records having different violation counts

Query:

```
CCREATE TABLE schema_violations (
   schema_id INT PRIMARY KEY,
   schema_name VARCHAR(30),
   violation_count INT
```

```
);

INSERT INTO schema_violations VALUES
    (1, 'Aarav',   0),
    (2, 'Mehul',   2),
    (3, 'Karan',   5),
    (4, 'Ishita',  9),
    (5, 'Nisha',  15);
select * from schema_violations
```

Output:

schema_id	schema_name	violation_count
1	Aarav	0
2	Mehul	2
3	Karan	5
4	Ishita	9
5	Nisha	15

Step 2: Classifying Data Using CASE Expression

- Retrieve schema names and their violation counts.
- Use conditional logic to classify each schema into categories such as:
    - No Violation
    - Minor Violation
    - Moderate Violation
    - Critical Violation

Query:

```
SELECT
    schema_name,
```

```
    violation_count,
    CASE
        WHEN violation_count = 0          THEN 'No Violation'
        WHEN violation_count BETWEEN 1 AND 3 THEN 'Minor Violation'
        WHEN violation_count BETWEEN 4 AND 7 THEN 'Moderate Violation'
        ELSE                              'Critical Violation'
    END AS violation_status
FROM schema_violations;
```

Output:

schema_name	violation_count	violation_status
Aarav	0	No Violation
Mehul	2	Minor Violation
Karan	5	Moderate Violation
Ishita	9	Critical Violation
Nisha	15	Critical Violation

Step 3: Applying CASE Logic in Data Updates

- Add a new column to store approval status.
- Update this column based on violation count using conditional rules such as:
    - Approved
    - Needs Review
    - Rejected

Query:

```
ALTER TABLE schema_violations
ADD COLUMN approval_status VARCHAR(20);
```

```
UPDATE schema_violations
SET approval_status =
   CASE
      WHEN violation_count = 0  THEN 'Approved'
      WHEN violation_count <= 5 THEN 'Needs Review'
      ELSE                           'Rejected'
   END;

SELECT * FROM schema_violations;
```

Output:

schema_id	schema_name	violation_count	approval_status
1	Aarav	0	Approved
2	Mehul	2	Needs Review
3	Karan	5	Needs Review
4	Ishita	9	Rejected
5	Nisha	15	Rejected

Step 4: Implementing IF–ELSE Logic Using PL/pgSQL

- Use a procedural block instead of a SELECT statement.
- Declare a variable representing violation count.
- Display different messages based on the value of the variable using IF–ELSE logic.

Query:

```
DO $$
DECLARE
   v_count INT := 6;
BEGIN
```

```
    IF v_count = 0 THEN
        RAISE NOTICE 'No violations detected';
    ELSIF v_count <= 5 THEN
        RAISE NOTICE 'Minor violations – review required';
    ELSE
        RAISE NOTICE 'Critical violations – access denied';
    END IF;
END $$;
```

Output:

```
NOTICE:  Critical violations - access denied
```

Step 5: Real-World Classification Scenario (Grading System)

- Create a table to store student names and marks.
- Classify students into grades based on their marks using conditional logic.

Query:

Step 5.1: Create Table

```
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(30),
    marks INT
);
```

Insert Sample Data

```
INSERT INTO students VALUES
    (1, 'Aarav', 88),
    (2, 'Mehul', 74),
```

```
(3, 'Karan', 61),
(4, 'Ishita',46),
(5, 'Nisha', 32);
```

Output:

student_id	student_name	marks
1	Aarav	88
2	Mehul	74
3	Karan	61
4	Ishita	46
5	Nisha	32

Step 5.2: Classify Students Using Conditional Logic

```
SELECT
    student_name,
    marks,
    CASE
        WHEN marks >= 80 THEN 'A'
        WHEN marks >= 70 THEN 'B'
        WHEN marks >= 60 THEN 'C'
        WHEN marks >= 40 THEN 'D'
        ELSE                  'F'
    END AS grade
FROM students;
```

Output:

student_name	marks	grade
Aarav	88	A
Mehul	74	B
Karan	61	C
Ishita	46	D
Nisha	32	F

Step 6: Using CASE for Custom Sorting

- Retrieve schema details.
- Apply conditional priority while sorting records based on violation severity.

Querr:

```
SELECT
   schema_name,
   violation_count
FROM schema_violations
ORDER BY
   CASE
      WHEN violation_count = 0  THEN 1
      WHEN violation_count <= 3 THEN 2
      WHEN violation_count <= 7 THEN 3
      ELSE                      4
   END;
```

Output:

schema_name	violation_count
Aarav	0
Mehul	2
Karan	5
Ishita	9
Nisha	15

Learning Outcome:

This experiment demonstrates how conditional logic is implemented in PostgreSQL using **CASE expressions** and **IF–ELSE constructs**.
Students gain strong command over **rule-based SQL logic**, which is essential for:

- Backend systems
- Analytics
- Compliance reporting
- Placement and technical interviews